

Módulo de Tickets e Impresión Térmica (ESC/POS)

Documentación técnica de la capa de impresión de Casa Pestalozzi POS. Enfoque: ~80% técnico · ~20% negocio. Última actualización: 2026-07-04.

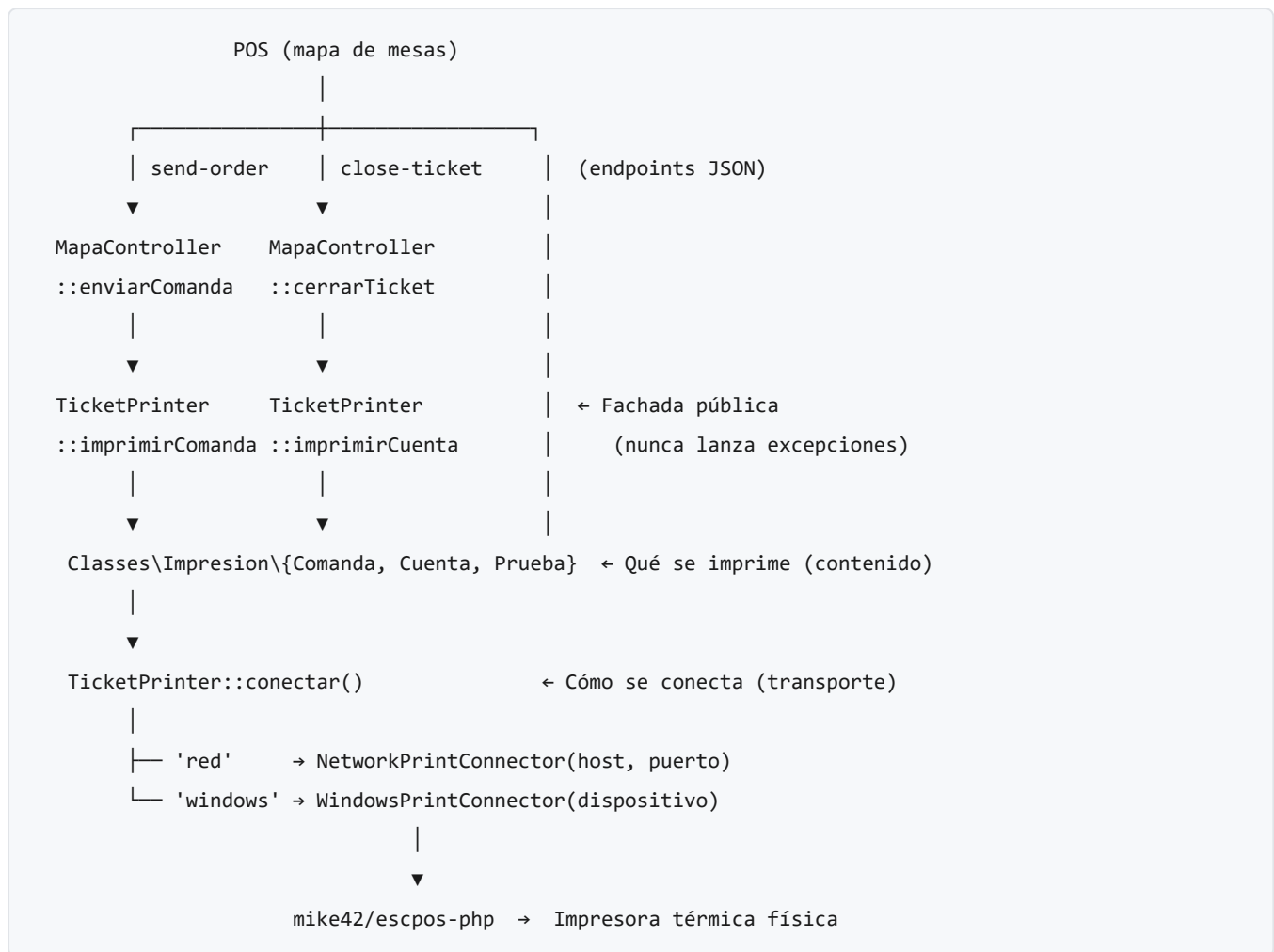
1. Qué resuelve (visión de negocio)

En el restaurante hay dos documentos físicos que se imprimen en papel térmico:

Documento	Para quién	Cuándo se imprime	Impresora
Comanda	Cocina / barra / café	Al enviar la orden a producción	Una por área (rol comanda)
Cuenta	Cliente (ticket de cobro)	Al cobrar / cerrar la mesa	Una sola de caja (rol cuenta)

Regla de oro del negocio: **la impresión nunca debe frenar la operación**. Si una impresora está apagada, sin papel o desconectada, el mesero debe poder seguir enviando órdenes y cobrando. Por eso toda la capa de impresión es un *efecto secundario*: falla en silencio (se registra en log) y jamás interrumpe el cobro, el envío de comandas ni el token de feedback del cliente.

2. Arquitectura general



Separación de responsabilidades:

- **Classes\TicketPrinter** — fachada. Decide *qué* imprimir y *en qué* impresora (consultando el modelo), aísla errores y expone `imprimirComanda`, `imprimirCuenta`, `imprimirPrueba`, `ultimoError`.
- **Classes\Impresion\Documento** (base) + `Comanda`, `Cuenta`, `Prueba` — definen el *contenido* y *formato* del papel (encabezados, columnas, corte).
- **Model\Impresora** — registro de impresoras y *routing* por rol/área.
- **Controllers\AdminPrintersController** + vistas `views/admin/printers/*` — CRUD para dar de alta impresoras y lanzar tickets de prueba.
- **mike42/escpos-php** (vendedor) — genera los bytes ESC/POS y provee los *connectors* de transporte.

3. Componentes y ficheros

Ruta	Rol
classes/TicketPrinter.php	Fachada de impresión. Punto de entrada único.
classes/Impresion/Documento.php	Clase base: helpers de formato (<code>separador()</code> , <code>dosColumnas()</code> , <code>ancho</code>).
classes/Impresion/Comanda.php	Ticket de cocina/barra por área.
classes/Impresion/Cuenta.php	Ticket de cobro del cliente.
classes/Impresion/Prueba.php	Ticket de prueba de conectividad.
models/Impresora.php	Modelo + validación + routing (<code>comandaPorArea</code> , <code>cuenta</code>).
controllers/AdminPrintersController.php	CRUD <code>/admin/printers</code> + prueba.
controllers/MapaController.php	Dispara comanda (<code>enviarComanda</code>) y cuenta (<code>cerrarTicket</code>).
views/admin/printers/{index,form}.php	UI de administración de impresoras.
database/ddl.sql · database/dml.sql	Esquema (tabla <code>impresoras</code>) y datos de siembra.

4. Modelo de datos: tabla `impresoras`

```
CREATE TABLE impresoras (  
  id          INT AUTO_INCREMENT PRIMARY KEY,  
  nombre      VARCHAR(60) NOT NULL,  
  area_id     TINYINT UNSIGNED NULL,      -- NULL = impresora de cuenta/caja  
  rol         ENUM('comanda','cuenta') NOT NULL DEFAULT 'comanda',  
  conexion    ENUM('red','windows') NOT NULL DEFAULT 'red',  
  host        VARCHAR(64) NOT NULL,      -- sólo aplica a conexion='red'  
  puerto      INT NOT NULL DEFAULT 9100,  -- sólo aplica a conexion='red'  
  dispositivo VARCHAR(120) NULL,         -- windows: nombre de impresora o smb://host/recurso  
  ancho       TINYINT NOT NULL DEFAULT 48, -- 48 = papel 80mm · 32 = papel 58mm  
  activo      TINYINT(1) NOT NULL DEFAULT 1,  
  FOREIGN KEY (area_id) REFERENCES areas_produccion(id)  
);
```

Esquema (DDL) y datos (DML)

La base se define en dos archivos: `database/ddl.sql` (estructura) y `database/dml.sql` (datos de siembra). La tabla `impresoras` — con sus columnas `conexion` (`red` / `windows`) y `dispositivo` — se crea directamente en el `CREATE TABLE` del DDL, sin `ALTER TABLE` posteriores.

Al desplegar en una máquina nueva, ejecuta primero `database/ddl.sql` y luego `database/dml.sql`. Sin la columna `conexion` el formulario y `conectar()` fallan.

Campos según el tipo de conexión

Campo	red	windows
host	☒ obligatorio	— (se guarda '')
puerto	☒ (def. 9100)	— (def. 9100)
dispositivo	— (NULL)	☒ nombre o <code>smb://host/recurso</code>

El controlador **normaliza** los campos del modo no elegido para no arrastrar datos viejos (`AdminPrintersController::asignarDatos`), y el formulario **deshabilita** por JS los inputs de los bloques ocultos para que no se envíen.

5. Routing de impresoras (a qué impresora va cada documento)

El modelo resuelve el destino a partir del **rol** y el **área**:

```
// Comanda: impresora ACTIVA de rol 'comanda' asignada a esa área de producción.
Impresora::comandaPorArea($area_id); // WHERE rol='comanda' AND area_id=? AND activo=1

// Cuenta: primera impresora ACTIVA de rol 'cuenta'.
Impresora::cuenta(); // WHERE rol='cuenta' AND activo=1
```

Áreas de producción (`AdminPrintersController::AREAS`):

area_id	Área
1	Café
2	Jugos
3	Cocina
4	Horno

Reglas de validación (`Impresora::validar`):

- Una impresora de **comanda** *debe* tener `area_id`.
- Una impresora de **cuenta** *no* lleva área (`area_id = NULL`).
- `conexion='red'` exige `host` y `puerto` (1–65535).
- `conexion='windows'` exige `dispositivo`.

Si no hay impresora activa que cumpla el criterio, `TicketPrinter` registra el caso en `error_log` y devuelve `false` / `[area=>false]`, **sin lanzar excepción**.

6. Flujo operativo end-to-end

1. Abrir cuenta	POST /admin/api/open-ticket	→ no imprime
2. Enviar comanda	POST /admin/api/send-order	→ imprime COMANDA por área
3. Cobrar / cerrar mesa	POST /admin/api/close-ticket	→ imprime CUENTA (cobro)

6.1 Comanda (`MapaController::enviarComanda`)

- Inserta en `ticket_items` **sólo los productos de esta tanda** y los agrupa por `area_id` (default `3` = Cocina si no viene el área).
- Llama `TicketPrinter::imprimirComanda($itemsPorArea, $meta)`, que imprime **una comanda por área** en la impresora de esa área.
- Sólo se imprime lo recién enviado: un re-envío del ticket **no** reimprime comandas anteriores.
- La impresión va en su propio `try/catch`; el resultado (`print_ok`) es meramente informativo y **nunca** cambia la respuesta `ok` del endpoint.

6.2 Cuenta (`MapaController::cerrarTicket`)

- Valida `metodo_pago` $\in \{ \text{efectivo}, \text{tarjeta} \}$.
- Marca el ticket `cerrado`, guarda el método de pago y genera el token de feedback.
- Lee los items **no cancelados** y llama `TicketPrinter::imprimirCuenta(...)`.
- Igual que la comanda: en su propio `try/catch`; un fallo de impresora no revierte el cierre ni el token.

Nota de negocio — no existe "pre-cuenta". Hoy la cuenta se emite únicamente en el cierre/cobro. Si la operación requiere entregar la cuenta al cliente *antes* de pagar, hay que agregar un endpoint que reutilice `imprimirCuenta()` sin cambiar el estado del ticket (mejora pendiente, no implementada).

7. Transporte: cómo se conecta a la impresora

`TicketPrinter::conectar(Impresora $i): PrintConnector` elige el connector según `$i->conexion`:

```
switch ($impresora->conexion) {
    case 'windows': return new WindowsPrintConnector((string)$impresora->dispositivo);
    default:        return new NetworkPrintConnector($impresora->host, (int)$impresora->puerto);
}
```

El ciclo de envío (`TicketPrinter::enviar`) abre el connector, escribe el documento, y **siempre** cierra (`$printer->close()`), que finaliza el connector). Si algo falla, guarda el mensaje en `TicketPrinter::$ultimoError` (expuesto por `ultimoError()`) y lo escribe en `error_log` .

7.1 Red (TCP 9100) — `NetworkPrintConnector`

- Impresora térmica con puerto Ethernet/Wi-Fi e IP fija.
- `host` = IP (ej. `192.168.1.50`), `puerto` = `9100` (RAW/JetDirect estándar).
- **La opción más estable** para producción: no depende del SO ni del spooler.
- Requiere IP fija (reserva por DHCP o IP estática en la impresora).

7.2 Windows por nombre / spooler — `WindowsPrintConnector`

Impresora instalada en Windows (típicamente USB térmica) y **compartida**. Este modo tiene un comportamiento clave que hay que entender:

- Si se pasa un **nombre pelado** (ej. `POS80C`), el connector lo trata como impresora **de red** (`isLocal = false`) aunque esté en la misma máquina: al finalizar, escribe un archivo temporal y hace `copy(tmp, "\\NOMBRE-PC\POS80C")` .
- Por lo tanto `POS80C` **se usa como NOMBRE DEL RECURSO COMPARTIDO**, no como el nombre "bonito" de la impresora. Deben coincidir exactamente.
- También se acepta la notación explícita `smb://NOMBRE-PC/POS80C` .
- Sólo los puertos locales `COM1` / `LPT1` entran por escritura directa; una USB no encaja ahí, por eso se usa el truco de compartir.

8. ⚠ Consideraciones al conectar y registrar impresoras (LEER ANTES DE INSTALAR)

8.1 Requisito de entorno — extensión `intl` de PHP (crítico)

`escpos-php` usa `IntlBreakIterator` para segmentar el texto al hacer `$printer->text()` . **Sin la extensión `intl` cargada, toda impresión revienta** con `Class "IntlBreakIterator" not found` — antes siquiera de tocar la impresora.

Verificar y habilitar:

```
php -m | grep -i intl          # ¿aparece "intl"?
php --ini                     # localizar el php.ini cargado
```

Si no aparece, en el `php.ini` correspondiente descomentar:

```
extension=intl
```

y **reiniciar el servidor** (`php -S` , Apache, etc.) para que tome el cambio. El DLL `php_intl.dll` ya viene en `ext/` en los builds estándar de Windows.

Este fue el motivo real por el que "no conectaba" durante las pruebas: el error de la impresora era un síntoma; la causa era `int1` deshabilitado. El detalle real ahora se muestra en la alerta de la prueba (ver §9).

8.2 Ancho de papel

- **80 mm** → `ancho = 48` columnas.
- **58 mm** → `ancho = 32` columnas.

Un ancho mal configurado descuadra columnas y corta texto. Es un campo por impresora, no global.

8.3 Impresoras de RED

- Asignar **IP fija** (o reserva DHCP). Si la IP cambia, deja de imprimir.
- Puerto `9100` salvo que el fabricante indique otro.
- Verificar que POS e impresora estén en la **misma red/segmento** y que ningún firewall bloquee el 9100.

8.4 Impresoras por NOMBRE de Windows (spooler)

Lista de verificación cuando registres una tipo `windows`:

1. **Nombre del recurso compartido exacto.** *Impresoras y escáneres* → (*impresora*) → *Propiedades de impresora* → *pestaña Compartir* → "Nombre del recurso compartido". Ese texto exacto es el que va en el campo `dispositivo` (o usar `smb://NOMBRE-PC/RECURSO`). El nombre de la impresora y el del recurso pueden diferir.
2. **Uso compartido habilitado** en esa impresora.
3. **Permisos del proceso.** El connector hace un `copy()` al recurso `\\NOMBRE-PC\RECURSO`; el usuario que ejecuta PHP necesita acceso:
 - Con `php -S` lanzado por el usuario interactivo: hereda sus permisos .
 - Con Apache/XAMPP como **servicio** (cuenta `SYSTEM / LocalService`): suele **no** tener acceso al recurso compartido → `Failed to copy file to printer`. Solución: correr Apache con la cuenta del usuario (`services.msc` → *Apache* → *Iniciar sesión* → *Esta cuenta*) o usar impresora de **red** en su lugar.
4. **Driver que respete el RAW ESC/POS.** Si "conecta" pero salen **caracteres basura**, el driver está reinterpretando los bytes: comparte la impresora con el driver "**Generic / Text Only**". (Esto es problema de *contenido*, no de *conexión*.)
5. Prueba rápida de aislamiento (como usuario, en `cmd`):

```
copy /b prueba.txt \\%COMPUTERNAME%\POS80C
```

Si esto imprime pero el POS no, el culpable es la cuenta del proceso PHP.

9. Probar la conexión (sin abrir mesas)

/admin/printers → botón **"Imprimir prueba"** en cada impresora. Envía un ticket de `Classes\Impresion\Prueba` a esa impresora concreta (ignora rol y estado `activo`), ideal para validar cableado y drivers.

Si falla, la alerta muestra una sugerencia por tipo de conexión **y el detalle real** de la excepción subyacente:

```
No se pudo imprimir en POS80C. <sugerencia> • Detalle: <mensaje real>
```

Ese `Detalle:` proviene de `TicketPrinter::ultimoError()` y es la herramienta #1 para diagnosticar (ej. `Failed to copy file to printer`, `Class "IntlBreakIterator" not found`, `timeout de socket`, etc.).

10. Diagnóstico rápido (troubleshooting)

Síntoma / <code>Detalle:</code>	Causa probable	Acción
<code>Class "IntlBreakIterator" not found</code>	Extensión <code>intl</code> deshabilitada	Habilitar <code>extension=intl</code> y reiniciar (§8.1)
<code>Failed to copy file to printer</code>	Recurso compartido inexistente o sin permiso	Verificar nombre del recurso y cuenta del proceso (§8.4)
<code>Printer 'X' is not a valid printer name</code>	Nombre inválido para <code>WindowsPrintConnector</code>	Usar nombre del recurso o <code>smb://host/recurso</code>
Imprime caracteres basura	Driver no RAW	Compartir con driver <i>Generic / Text Only</i> (§8.4.4)
Comanda no sale	Sin impresora <code>comanda</code> activa para esa área	Registrar/activar impresora del área (§5)
Cuenta no sale al cobrar	Sin impresora <code>cuenta</code> activa	Registrar POS80C con rol Cuenta y activa (§5)
Timeout / conexión rechazada (red)	IP/puerto/red incorrectos	Verificar IP fija, puerto 9100, misma red (§8.3)

Los mensajes completos siempre quedan en el `error_log` de PHP con el prefijo `TicketPrinter - ...`.

11. Extender el módulo (para desarrolladores)

- **Nuevo documento** (ej. corte de caja): crear una clase en `Classes\Impresion` que extienda `Documento` e implemente `imprimir(Printer $printer)`, y exponer un método en `TicketPrinter` que resuelva la impresora y llame a `enviar()`.
- **Nuevo tipo de conexión:** agregar el valor al `ENUM conexion` (migración), ampliar la validación en `Impresora::validar`, el `switch` de `TicketPrinter::conectar`, `asignarDatos / guardar` del controlador y los bloques del formulario. Ese es exactamente el patrón que siguió `windows`.

- **Regla invariable:** la impresión es efecto secundario. Cualquier método nuevo debe capturar `\Throwable` , registrar en log y **jamás** propagar la excepción al flujo del POS.

